

Algorithms and Data Structures

Problem Sheet 8

Hermann Schichl

73. Draw the red-black trees that result after successively inserting the keys 41; 38; 31; 12; 19; 8 into an initially empty red-black tree.
74. Consider a red-black tree formed by inserting n nodes with RB-INSERT. Argue that if $n > 1$, the tree has at least one red node.
75. Argue that after RB-DELETE-FIXUP executes, the root of the tree must be black.
76. Argue that if in RB-DELETE both x and $x.p$ are red, then property 4 is restored by the call to RB-DELETE-FIXUP(T, x).
77. Professor Baron worries that at the start of case 1 of RB-DELETE-FIXUP, the node $x.p$ might not be black. If $x.p$ is not black, then lines 5–6 are wrong. Show that $x.p$ must be black at the start of case 1, so that the professor does not need not be concerned.
78. A node x is inserted into a red-black tree with RB-INSERT and then is immediately deleted with RB-DELETE. Is the resulting red-black tree always the same as the initial red-black tree? Justify your answer.
79. Consider the operation RB-ENUMERATE(T, r, a, b), which outputs all the keys k such that $a \leq k \leq b$ in a subtree rooted at node r in an n -node red-black tree T . Describe how to implement RB-ENUMERATE in $\Theta(m + \lg n)$ time, where m is the number of keys that are output. Assume that the keys in T are unique and that the values a and b appear as keys in T . How does your solution change if a and b might not appear in T ?
80. An AVL tree is a binary search tree that is height balanced: for each node x , the heights of the left and right subtrees of x differ by at most 1. To implement an AVL tree, maintain an extra attribute h in each node such that $x.h$ is the height of node x . As for any other binary search tree T , assume that $T.root$ points to the root node.
 - (a) Prove that an AVL tree with n nodes has height $O(\lg n)$. (*Hint: Prove that an AVL tree of height h has at least F_h nodes, where F_h is the h th Fibonacci number.*)
 - (b) To insert into an AVL tree, first place a node into the appropriate place in binary search tree order. Afterward, the tree might no longer be height balanced. Specifically, the heights of the left and right children of some node might differ

- by 2. Describe a procedure **BALANCE**(x), which takes a subtree rooted at x whose left and right children are height balanced and have heights that differ by at most 2, so that $|x.right.h - x.left.h| \leq 2$, and alters the subtree rooted at x to be height balanced. The procedure should return a pointer to the node that is the root of the subtree after alterations occur. (*Hint: Use rotations.*)
- (c) Using part (b), describe a recursive procedure **AVL-INSERT**(T, z) that takes an AVL tree T and a newly created node z (whose key has already been filled in), and adds z into T , maintaining the property that T is an AVL tree. As in **TREE-INSERT**, assume that $z.key$ has already been filled in and that $z.left = NIL$ and $z.right = NIL$. Assume as well that $z.h = 0$.
- (d) Show that **AVL-INSERT**, run on an n -node AVL tree, takes $O(\lg n)$ time and performs $O(\lg n)$ rotations.